SQL SERVER / T-SQL

Fundamentos de programação de banco de dados

Integridade Referencial

Prof. Me. Francisco Martins

INTEGRIDADE REFERENCIAL

- **Integridade referencial é o princípio que garante que os relacionamentos entre tabelas em um banco de dados sejam consistentes, evitando referências a registros inexistentes.**

- A **integridade referencial** assegura que uma **chave estrangeira (FK)** em uma tabela sempre aponte para uma **chave primária (PK)** existente em outra tabela, evitando registros órfãos ou inconsistentes. Em outras palavras, um registro dependente não pode existir sem que o registro de referência exista, garantindo que os dados estejam corretos e completos.

INTEGRIDADE REFERENCIAL

Funcionamento:

- **Chave Primária (PK)**: identifica de forma única cada registro em uma tabela. Não pode conter valores nulos e deve ser única.
- **Chave Estrangeira (FK)**: faz referência a uma PK em outra tabela, estabelecendo a dependência entre registros.

*Por exemplo, em um banco de dados de locadora, se a tabela **Carros** possui uma **FK** apontando para a tabela **Proprietários**, não é possível inserir um carro com um **ID** de proprietário que não exista, nem excluir um proprietário que ainda possua carros registrados.*

INTEGRIDADE REFERENCIAL

Regras e Operações

- A integridade referencial é aplicada durante operações de **inserção, atualização e exclusão**:
  - **Insert**: Não permite adicionar um registro com **FK** que não corresponda a uma **PK** existente.
  - **Update**: Não permite alterar a **FK** ou **PK** de forma que quebre a relação existente.
  - **Delete**: Não permite remover um registro **"pai"** se houver registros **"filhos"** dependentes, a menos que sejam tratados previamente (como exclusão em cascata ou atualização de **FK**).

INTEGRIDADE REFERENCIAL

Importância

- Garantir integridade referencial é essencial para:
  - Evitar **dados inconsistentes** ou órfãos.
  - Manter **confiabilidade** em análises e relatórios, especialmente em sistemas de BI(Business Intelligence)e OLAP(OnLine Analitics Processing) com grandes quantidades de dados.
  - Prevenir **erros lógicos** em aplicações que dependem de relacionamentos corretos entre tabelas.

INTEGRIDADE REFERENCIAL

Exemplo Prático

- **1:N (um para muitos)**: Um **Funcionário** pode ter vários **Dependentes**, mas cada **Dependente** pertence a apenas um **Funcionário.** A **FK** do **Funcionário** na tabela **Funcionário,** garante que não existam **Dependentes** sem **Funcionário** associado.
- Em resumo, a integridade referencial é um mecanismo fundamental para **manter a consistência e confiabilidade dos dados** em bancos relacionais, sendo implementada por meio de **chaves primárias e estrangeiras** e respeitada pelo **SGBD** durante todas as operações de manipulação de dados.

EXEMPLO PRÁTICO

- Utilize o SQL-SERVER e crie o banco de dados “EMPRESA”, e ative o banco com o comando USE como abaixo:

```
CREATE DATABASE EMPRESA;

USE EMPRESA;

GO
```

EXEMPLO PRÁTICO

- Em seguida, crie as tabelas Funcionário e Dependente como apresentado ao lado:

```
create table funcionario(
IdFuncionario smallint not null IDENTITY(1,1) PRIMARY KEY,
NomeFuncionario varchar(30) );

create table dependente(
IdDependente smallint not null IDENTITY(1,1) PRIMARY KEY,
NomeDependente varchar(30),
IdFuncDep smallint,
foreign key(IdFuncDep)
references funcionario(IdFuncionario)
on delete cascade );
```

EXEMPLO PRÁTICO

Em seguida, insira três Funcionários com **INSERT INTO**:

Maria, João e Pedro.

Após, verifique se realmente foram inseridos na tabela funcionário com o comando **SELECT**.

```
28    insert into funcionario (NomeFuncionario)
29    values ('Maria'), ('João'), ('Pedro');
30
31    Select * from funcionario;
32
```

% 2 0

Resultados | Mensagens

IdFuncionario	NomeFuncionario
1	Maria
2	João
3	Pedro

EXEMPLO PRÁTICO

Em seguida, insira os registros de dependentes com **INSERT INTO** como abaixo: Dois para Maria, Dois para João e Um para Roberta.

Após, verifique se realmente os dependentes foram inseridos na tabela dependente com o comando **SELECT**.

```
33  insert into dependente (NomeDependente, IdFuncDep)
34  Values ('Carla', 1), ('Joaquim', 1),('Paulo', 2),('Luiz', 2),('Roberta', 3);
35
36  --Mostrar os dependentes e a FK que vincula ao Funcionario
37  select * from dependente;
38
```

2 0 Ln: 37, Car: 1

Resultados | Mensagens

IdDependente	NomeDependente	IdFuncDep
1	Carla	1
2	Joaquim	1
3	Paulo	2
4	Luiz	2
5	Roberta	3

EXEMPLO PRÁTICO

Para mostrar os dependentes atrelados aos funcionários através da junção das tabelas com a cláusula INNER JOIN.

```
40    select IdFuncionario, NomeFuncionario
41    as Funcionario, dependente.NomeDependente
42    as Dependente from funcionario
43    INNER JOIN dependente on IdFuncionario = dependente.IdFuncDep;
```

% 2 0

Resultados | Mensagens

IdFuncionario	Funcionario	Dependente
1	Maria	Carla
1	Maria	Joaquim
2	João	Paulo
2	João	Luiz
3	Pedro	Roberta

EXEMPLO PRÁTICO

A **integridade referencial** é demonstrada quando da **exclusão** de um funcionário. Automaticamente serão **excluídos** também os dependentes dele.

Neste caso excluímos o funcionário João cujo código era igual a **2.** Assim, seus dependentes **Paulo e Luiz** também foram para manter a **integridade**.

```
55   delete from funcionario where funcionario.IdFuncionario = 2;
56
57   select * from dependente;
58
```

2 0

Resultados | Mensagens

IdDependente	NomeDependente	IdFuncDep
1	Carla	1
2	Joaquim	1
5	Roberta	3

INTEGRIDADE REFERENCIAL

O comando **ON DELETE RESTRICT** em **SQL** é uma restrição de integridade referencial que impede a exclusão de uma linha pai se ainda houver linhas filhas dependentes. Isso significa que, ao tentar excluir um registro pai, o banco de dados rejeitará a exclusão se houver registros filhos que ainda fazem referência a ele. Essa restrição é especialmente útil em cenários onde a perda acidental de dados é uma preocupação, como em sistemas de gestão de recursos humanos onde os colaboradores estão vinculados a organogramas.